Exerciții – Partea 1

1. Scrieți un program în C++ care să citească de la tastatură două numere întregi a și b și să afișeze suma lor, $a + b$.

   Referințe utile: structura unui program C++, declararea de variabile, operații de citire și de afișare.

2. Scrieți un program în C++ care să citească de la tastatură un număr natural n și apoi să afișeze, pe câte un rând, toate numerele naturale de la 1 la n, cu mențiunea că numerele care sunt divizibile cu 3 vor fi înlocuite cu textul `Fizz`, numerele care sunt divizibile cu 5 vor fi înlocuite cu textul `Buzz`, iar numerele care sunt divizibile și cu 3 și cu 5 vor fi înlocuite cu textul `FizzBuzz`[1].

   Referințe utile: `for` loop, structură condiționată `if...else`, operatorul modulo (%).

3. Scrieți un program în C++ care să citească de la tastatură un număr întreg n, care să fie *natural*, *nenul* și *par*.

   Programul îi va cere utilizatorului să introducă la tastatură numărul, va citi un număr întreg și va continua să facă acest lucru până când numărul citit respectă cerințele impuse (să fie > 0 și divizibil cu 2).

   La final, afișați pe ecran valoarea lui $n/2$.

   Observație: puteți folosi acest pattern când aveți nevoie să citiți date de la tastatură care să respecte un anumit format.

   Referințe utile: structură repetitivă `do...while`

4. Limbajul C++ permite *supraîncărcarea* funcțiilor: definirea mai multor subprograme care au *aceeași denumire*, dar diferă prin *tipul* sau *numărul* parametrilor pe care îi acceptă.

   Implementați mai multe funcții, toate denumite `absolute_value`, care să calculeze valoarea absolută a unui:

   - Număr întreg (un parametru de tip `int`);
   - Număr rațional (un parametru de tip `double`);
   - Număr complex (doi parametrii de tip `double`, care reprezintă partea lui reală și partea lui imaginară)

[1] Aceasta este celebra problemă *fizz buzz*, despre care se spunea că e o problemă tipică de interviu de angajare ca programator.

- Vector din $\mathbb{R}^3$ (trei parametrii de tip `double`, care reprezintă coordonatele vectorului).

Antetele lor ar trebui să fie:

```
double absolute_value(int nr);
double absolute_value(double nr);
double absolute_value(double real, double imag);
double absolute_value(double x, double y, double z);
```

Apelați fiecare dintre aceste funcții din subprogramul principal. Observați cum compilatorul este capabil să determine automat la care variantă a funcției vă referiți.

Referințe utile: supraîncărcarea funcțiilor în C++, `std::sqrt`;

5. Implementați o funcție în C++ care să interschimbe valorile a doi parametri, numere întregi, **transmiși prin referință**.

   Antetul funcției ar trebui să fie: `void interschimba(int& a, int& b);`

   Scrieți și un cod corespunzător în subprogramul principal, care să apeleze acest subprogram și să verifice că funcționează cum trebuie.

   Referințe utile: interschimbarea a două numere întregi, referințe în C++.

6. Implementați o funcție în C++ care să interschimbe valorile către care trimit cei doi parametri de tip pointer la numere întregi (`int*`).

   Pentru a obține un pointer care să trimită la o variabilă deja declarată, putem folosi operatorul „address-of” (ampersandul `&`). Pentru a citi valoarea din zona de memorie indicată de către un pointer (presupus valid), putem folosi operatorul de dereferențiere (asteriscul `*`).

   Antetul funcției ar trebui să fie: `void interschimba(int* a, int* b);`

   O puteți apela în `main` în felul următor:

```
int x = 1, y = 2;
interschimba(&x, &y);
// x ar trebui sa aiba acum valoarea 2,
// iar y valoarea 1
```

Observație: în limbajul C nu există referințe. În acel caz, pointerii sunt singurul mod prin care ne putem referi cu mai multe nume la aceeași zonă de memorie.

Referințe utile: utilizarea pointerilor în C++, operatorii de referențiere și de dereferențiere, intereschimbarea a două numere folosind pointeri.

7. Scrieți un program în C++ care să citească de la tastatură un număr natural n și apoi n numere întregi, pe care să le salveze într-un vector alocat dinamic. Îl puteți crea scriind `int* vector = new int[n];` după ce l-ați citit pe n.

   Sortați vectorul folosind funcția `std::sort` din biblioteca standard și apoi afișați rezultatul pe ecran. Va fi nevoie să includeți header-ul `algorithm`, i.e. `#include <algorithm>`.

   Funcția `sort` primește doi parametrii: un pointer către primul element din vector, respectiv un pointer către elementul care ar veni fix după ultimul element din vector (nu e o problemă că acesta nu există).

   Apelul ar trebui să fie: `sort(&vector[0], &vector[n]);`

   La final, eliberați memoria alocată pentru vector prin `delete[] vector;` (trebuie să folosiți `delete[]`, nu doar `delete` simplu, pentru că este vorba de un întreg vector de elemente).

   Referințe utile: alocarea dinamică de memorie în C++, cum se utilizează `std::sort`.

8. Definiți o nouă structură `Complex` (folosiți cuvântul cheie `struct`), care să rețină **partea reală** și **partea imaginară** a unui număr complex (numere reale stocate ca `double`).

   Referințe utile: cum se declară o structură în C++.

   Implementați următoarele funcționalități pentru aceasta:

   - Un constructor fără parametrii, care să instanțieze numărul complex 0.

     Referințe utile: constructori.

   - Un constructor care primește ca parametrii partea reală și partea imaginară a unui număr complex și le salvează în variabilele membru corespunzătoare. Folosiți o listă de inițializare în constructor pentru a face acest lucru.

     Observație: în acest caz, nu este nevoie să folosim liste de inițializare. Dar este bine să vă familiarizați cu sintaxa lor, deoarece este singura posibilitate pentru a inițializa variabile membru care sunt `const`, care sunt referințe sau care nu au un constructor fără parametru (e.g. dacă am fi avut în structura noastră o variabilă membru de tip `ifstream`).

     Referințe utile: ce sunt și cum se folosesc listele de inițializare în C++.

- O metodă statică `from_polar` care primește ca parametrii modulul și argumentul (unghiul) unui număr complex, construiește și returnează un nou obiect de tip `Complex` cu partea reală și partea imaginară corespunzătoare.

  Antetul acesteia ar trebui să fie:

  ```
  static Complex from_polar(double modulus, double angle)
  ```

  Iar ulterior o puteți apela în programul principal folosind sintaxa:

  ```
  Complex z = Complex::from_polar(2, 0.5);
  ```

  Reamintesc că un număr complex z se poate scrie în mod echivalent ca

  $$z = x + iy = r(\cos\theta + i\sin\theta)$$

  unde x este partea reală a numărului complex, y este partea lui imaginară, r este modulul numărului complex și θ este argumentul acestuia.

  Observație: această metodă este un exemplu al pattern-ului static factory method. Deja avem un constructor pentru clasă cu signatura

  ```
  Complex(double, double)
  ```

  așa că nu am fi putut defini încă unul care să primească doi parametri de tip `double`. *Static factories* ne permit să avem niște constructori „cu nume”.

  Referințe utile: variabile membru și metode statice, `<cmath>`.

- Supraîncărcați operatorul << pentru a permite afișarea unui număr complex, respectiv >> pentru a permite citirea de la tastatură a unui număr complex.

  Puteți găsi un exemplu de cum să faceți acest lucru aici. Antetul acestor funcții ar trebui să fie:

  ```
  1 ostream& operator<<(ostream& out, const Complex& c);
  2 istream& operator>>(istream& in, Complex& c);
  ```

  În cazul operatorului de afișare, este bine să primim obiectul de tip `Complex` printr-o referință constantă ca să evităm copierea lui și în același timp să nu-l modificăm din greșeală. În cazul operatorului de citire, trebuie să primiți obiectul prin referință ca să puteți să puteți actualiza datele din el.

  Operatorii ar putea fi definiți să returneze `void` în loc de `ostream&`

(respectiv `istream&`), dar atunci nu am mai putea înlănțui operațiile de citire/afișare (e.g. nu am putea scrie `cin >> a >> b >> c`).

După ce ați rezolvat acest subpunct, în programul principal ar trebui să puteți folosi operatorii ca în următorul exemplu:

```
1 Complex z;
2 cin >> z;
3 cout << z << '\n';
```

Observație: operatorii de citire/afișare pot fi supraîncărcați doar ca funcții în afara structurii/clasei; nu pot fi metode, deoarece vrem ca primul lor parametru să fie de tipul `istream&`/`ostream&`, nu obiectul implicit de tip `Complex` pe care îl primesc toate metodele.

Referințe utile: supraîncărcarea operatorilor <</>> în C++ (nu o să aveți nevoie de modificatorul `friend`, deoarece în acest caz datele membru sunt publice).

- Supraîncărcați operatorii `+`, `-`, `*`, respectiv `/`, ca să puteți aduna, scădea, înmulți și împărți numere complexe.

  Aceste funcții nu ar trebui să modifice numerele complexe pe care le primesc ca parametri, ci ar trebui să returneze niște obiecte noi.

  Observație: acești operatori îi puteți defini fie ca metode, caz în care vor primi un singur parametru de tip `Complex` (deoarece au acces deja la obiectul implicit), sau ca funcții libere, caz în care trebuie să primească doi parametrii de tip `Complex`.

  Referințe utile: supraîncărcarea operatorilor în C++.

- Supraîncărcați operatorii `==` și `!=` ca să puteți compara dacă două numere complexe sunt egale sau nu.

  Aceste funcții ar trebui să aibă un parametru de tip `Complex` (dacă sunt definite ca metode) sau doi parametri de tip `Complex` (dacă sunt definite ca funcții libere) și să returneze o valoare de tip `bool` (`true` sau `false`).

  Observație: dacă vreți să afișați rezultatul unei comparații, va trebui să scrieți `cout << (z1 == z2);` este obligatoriu să puneți paranteze, din cauza ordinii de evaluare a operatorilor în C++.

  Referințe utile: operatori de comparație, precedența operatorilor.

Scrieți subprogramul `main` corespunzător care să apeleze/testeze fiecare dintre metodele implementate.

9. Modificați structura definită la exercițiul anterior astfel încât să respecte *principiul encapsulării* (variabilele membru să fie private).

   Lăsați toate celelalte metode să fie publice. S-ar putea să fie nevoie să declarați cu modificatorul `friend` operatorii pe care i-ați supraîncărcat, ca să nu aveți erori de compilare.

   Adăugați metode de tip getter și setter pentru partea reală, respectiv partea imaginară a numărului complex.

   Referințe utile: modificatori de acces, funcții/clase „prieten” (modificatorul `friend`), exemplu de getter/setter pentru o variabilă membru privată.

10. Considerăm structura `Achizitie`, definită în felul următor:

```
1 struct Achizitie {
2     int id;
3     double valoare;
4 };
```

   Definiți o altă structură `ReferintaAchizitie`, care să aibă ca date membru:

   - un număr întreg constant, `const int id`;
   - o referință la o achiziție, `Achizitie& achizitie`;

   Puteți crea un obiect din această structură folosind constructorul fără parametrii?

   Definiți un constructor pentru această structură care să primească ca parametru un obiect de tip `Achizitie&`. Va trebui să folosiți o listă de inițializare ca să inițializați datele membru.

   Referințe utile: liste de inițializare în C++, inițializarea datelor membru constante, inițializarea datelor membru de tip referință.

11. În limbajul C, pentru a scrie un număr într-un fișier ar trebui să folosim următorul cod:

```
1 int numar = 1234;
2 FILE* f = fopen("fisier.txt", "w");
3 fprintf(f, "%d", numar);
4 fclose(f);
```

Observați că la final trebuie să apelăm `fclose` pentru a ne asigura că fișierul este salvat și închis cum trebuie. Dacă uităm să facem acest lucru, pot apărea bug-uri greu de diagnosticat.

Veți crea o clasă `MyFile` care să gestioneze o variabilă de tipul `FILE*`. Începeți prin a importa antetul `<cstdio>`, în care se află funcțiile menționate mai sus. Implementați:

- Un constructor care primește ca parametru un șir de caractere `const char nume_fisier[]`, reprezentând numele fișierului care va fi deschis. Acest constructor va folosi funcția `fopen` pentru a deschide fișierul indicat în modul de scriere și va salva pointerul returnat.

  Referințe utile: constructor cu parametri, `fopen`.

- Un destructor care apelează automat `fclose`.

  Referințe utile: destructor, `fclose`.

- O metodă `write` care primește ca parametru un număr întreg `numar` și îl scrie în fișier, lăsând un rând nou după, prin apelul `fprintf(f, "%d\n", numar);` (unde `f` este numele variabilei de tip `FILE*` din clasa voastră).

  Referințe utile: `fprintf`.

Creați o instanță a clasei `MyFile` în subprogramul principal și încercați să o folosiți ca să scrieți câteva valori într-un fișier.

Acest exercițiu este un exemplu foarte bun pentru utilitatea practică a constructorilor/destructorilor; vedeți și principiul *Resource Acquisition Is Initialization*.

12. Încercați să creați o copie a unei variabile de tip `MyFile` (clasa implementată la exercițiul anterior), de exemplu: `MyFile f("fisier.txt"); MyFile g(f);`

    Ce se întâmplă când se termină blocul în care sunt definite variabilele? Primiți o eroare?

    Constructorul de copiere generat implicit de compilator va copia variabila de tip `FILE*` în noua instanță a clasei. Când se rulează destructorii pentru ambele obiecte de tip `MyFile`, se va apela de două ori `fclose` pentru același fișier.

    Există moduri prin care putem elimina acest bug:

    - Definim constructorul de copiere, dar îl facem privat:

```
1  private:
2      MyFile(const MyFile&) {
3          // nu facem nimic aici
4      }
```

Asta va preveni copierea unui obiect de tip `MyFile` în afara clasei, dar în interiorul clasei încă există riscul să facem neintenționat acest lucru.

- (C++11) Folosim sintaxa „`= delete`” pentru a-i indica compilatorului că nu vrem să se genereze automat un constructor de copiere.

```
1      MyFile(const MyFile&) = delete;
```

Implementați o soluție asemănătoare și pentru operatorul =, pentru a preveni atribuirile de tipul `f = g;` pentru variabile de tip `MyFile` (acestea ar duce la același bug). Puteți fie să-l faceți privat, fie să folosiți sintaxa

```
1      void operator=(const MyFile&) = delete;
```

Referințe utile: ștergerea constructorilor de copiere și a operatorului =.

13. Nu vrem să permitem crearea unei copii a unui obiect de tipul `MyFile`, dar în unele cazuri ar fi util să putem transfera resursa deținută de el (variabila de tip `FILE*`) într-un alt obiect (de exemplu, ca să putem gestiona un vector de fișiere).

    În C++11, acest lucru se poate face definind constructorul/operatorul de mutare:

```
1  MyFile(MyFile&& other);
2  void operator=(MyFile&& other);
```

Aici apare un nou tip de referință (*`rvalue` reference*), care indică în fiecare caz că parametrul `other` a fost „preluat” de compilator și „mutat” în aceste funcții.

În cazul clasei noastre, fiecare dintre aceste funcții ar trebui să ia pointerul de tip `FILE*` din obiectul primit ca parametru, lăsându-l `NULL`/`nullptr` în urmă. Operatorul = ar trebui în plus să apeleze `fclose` pe pointerul pe care îl gestiona deja, dacă e cazul.

Va trebui să actualizați destructorul și metoda de afișare din `MyFile` ca să nu facă nimic în cazul în care variabila membru de tip `FILE*` este nulă.

Pentru a utiliza funcțiile nou-definite, putem folosi `std::move` din biblioteca `<utility>`. Exemplu de utilizare:

```
1 MyFile file1("..."), file2("...");
2 MyFile file3(move(file1));
3 file1 = move(file2);
```

`file3` ar trebui să scrie acum în fișierul deschis inițial de `file1`, `file1` ar trebui să scrie în fișierul deschis inițial de `file2`, iar `file2` ar trebui să nu scrie în niciun fișier (ar trebui să aibă nul pointerul la `FILE`).

Referințe utile: move semantics and rvalue references, rvalue references.

14. Definiți o clasă `IntVector` care să rețină un vector de numere întregi, alocat dinamic. Clasa ar trebui să aibă două variabile membru:

    - un număr întreg `int size`, reprezentând lungimea actuală a vectorului (cât de mare este zona de memorie indicată de `data`);
    - un pointer `int* data`, care reține vectorul de numere întregi, alocat dinamic (reține adresa returnată de `new []`).

    Implementați următoarele funcționalități:

    - Un constructor fără parametrii care să inițializeze pointer-ul `data` cu valoarea `NULL` sau (dacă folosiți C++11) `nullptr`, și să seteze `size` să fie 0.
    - Un constructor care permite crearea unui vector care constă din numărul întreg `x`, de `k` ori. Constructorul va avea ca parametrii `int k` și `int x`, în această ordine.

      Al doilea parametru ar trebui să aibă valoarea implicită 0.

      Constructorul va aloca dinamic memoria necesară pentru stocarea a `k` numere întregi și va seta toate elementele să aibă valoarea `x`.

      Referințe utile: alocarea dinamică de memorie în C++, setarea valorilor implicite pentru parametrii în C++.
    - Un destructor care dezalocă zona de memorie indicată de pointerul `data`, dacă acesta nu este `NULL`/`nullptr`.

      Referințe utile: destructor.
    - Un constructor de copiere care primește o referință constantă la un alt obiect din clasa `IntVector`, își alocă o nouă zonă de memorie (de

aceeași lungime cu cea a vectorului primit) și copiază în ea valorile din vectorul primit.

Antetul acestuia ar trebui să fie:

```
IntVector(const IntVector& v)
```

Observație: este obligatoriu să primim vectorul vechi prin referință (constantă). Dacă l-am primi prin valoare, la apel compilatorul ar folosi implicit constructorul de copiere, care apoi s-ar apela recursiv la infinit pe el însuși.

Referințe utile: constructorul de copiere în C++.

- Supraîncărcați operatorul = în așa fel încât să permită asignarea prin copiere a unui vector în alt vector.

  Acest operator ar trebui să copieze datele din vectorul primit ca parametru (la fel ca și constructorul de copiere), doar că mai întâi ar trebui să dezaloce memoria deținută de vectorul destinație (dacă este cazul, dacă acesta nu este vid).

  Antetul operatorului = ar trebui să fie:

  ```
  IntVector& operator=(const IntVector& v)
  ```

  Primim prin referință constantă parametrul ca să evităm crearea unei copii în plus a acestuia și pentru a nu-l modifica din greșeală. Mai mult, la final ar trebui să returnăm o referință la obiectul implicit (i.e. `return *this`), ca să permitem utilizarea sintaxei de atribuire multiplă (e.g. `v1 = v2 = v3`).

  Referințe utile: supraîncărcarea operatorului de atribuire/copiere.

(*) Definiți un constructor de mutare și supraîncărcați operatorul = de mutare ca să permiteți mutarea memoriei deținute de un vector într-un alt vector. Vectorul destinație ar trebui să primească lungimea și pointerul de la vectorul sursă, care va rămâne gol (va avea `size == 0` și `data == nullptr`).

- Supraîncărcați operatorul << ca să permiteți afișarea unui vector de numere întregi pe ecran (se vor afișa elementele acestuia, toate pe același rând, separate printr-un spațiu).
- Supraîncărcați operatorul >> ca să permiteți citirea unui vector de numere întregi dintr-un `istream`.

Operatorul va citi mai întâi *n*, numărul de elemente care urmează să fie citite, iar apoi cele *n* numere întregi.

Dacă vectorul primit ca parametru nu este gol, va trebui să eliberați mai întâi memoria reținută de el, iar apoi să-i alocați dinamic un array de dimensiune *n* în care să rețineți valorile citite.

- Definiți o metodă care să permită adăugarea unui nou element la sfârșitul vectorului (modificați vectorul actual, nu creați unul nou).

  Va trebui să alocați dinamic o nouă zonă de memorie de dimensiune suficientă, să copiați vechile elemente în ea, să adăugați noul element la finalul ei și apoi să eliberați memoria pentru vechiul tablou alocat dinamic (dacă este cazul).

15. Alocați dinamic un vector de obiecte de tip `IntVector` (i.e. un vector de vectori). Puteți face acest lucru folosind sintaxa

    ```
    IntVector* vv = new IntVector[10];
    ```

    Încercați să vedeți cum/dacă au fost inițializați acești vectori (inspectați-i în debugger, sau afișați lungimile lor). Ce constructor a fost folosit?

    De asemenea, observați că atunci când apelați `delete[]` pe acest vector de obiecte, compilatorul apelează automat destructorul pentru fiecare obiect (puteți confirma acest lucru punând un *breakpoint* în destructor și rulând codul cu un debugger, sau afișând un mesaj în destructor).

    Referințe utile: cum inițializează `new[]` obiectele alocate, `delete[]` apelează destructorul pentru fiecare obiect.

16. Ce se întâmplă dacă încercăm să folosim operatorul `=` de copiere, definit pentru clasa `IntVector`, în cazul în care obiectul destinație și obiectul sursă sunt același (de exemplu, dacă am scrie `v = v`)?

    Tratați și această posibilitate în implementarea voastră. Verificați dacă pointerul `this` este egal cu pointerul `&v`; în acest caz, nu este nevoie să alocați memorie sau să modificați datele membru.

    Referințe utile: tratarea cazului de *self assignment*.

17. Implementați o clasă `String`, care să rețină un șir de caractere alocat dinamic (i.e. un pointer la un tablou unidimensional de `char`, alocat dinamic).

    Puteți alege să rețineți doar pointerul către zona de memorie alocată dinamic, sau să rețineți și lungimea actuală a șirului de caractere (pentru a nu fi nevoie să o determinați tot timpul folosind `strlen`).

Observație: dacă vreți să creați un fișier header separat în care să declarați clasa, nu-l denumiți `String.h` pentru că s-ar putea să intre în conflict cu fișierul `string.h` din biblioteca standard. Puteți să-l denumiți în schimb `String.hpp`, de exemplu.

Referințe utile: tipuri de date fundamentale, tipul de date pointer, cum se declară o clasă, specificatori de acces;

Implementați următoarele funcționalități:

- Crearea unui șir de caractere vid (acest lucru va fi indicat prin setarea pointer-ului intern la `NULL` / `nullptr`, ca să nu alocăm degeaba un vector de caractere de lungime 0);

  Referințe utile: constructori, pointerul `NULL`, `nullptr`.

- Crearea unui șir de caractere pornind de la un string literal (să putem scrie `String sir("Un sir de caractere");`);

  Referințe utile: constructori cu parametri, string literals, operatorul `new[]`, `strlen`, `strncpy`;

- Eliberarea automată a memoriei alocate printr-un destructor;

  Referințe utile: operatorul `delete[]`;

- Copierea unui șir în alt șir; acest lucru ar trebui să se poată face atât prin constructorul de copiere, cât și prin operatorul =.

  Referințe utile: constructorul de copiere, operatorul de atribuire prin copiere, constructorul de copiere versus operatorul de atribuire;

(*) „Mutarea” unui șir în alt șir, noul șir preluând memoria gestionată de cel vechi, lăsând vid șirul sursă;

  Constructorul de mutare ar trebui să aibă signatura:

  `String(String&& other)`

  Iar operatorul = de mutare:

  `String& operator=(String&& other)`

  Referințe utile: move constructor;

- Adăugarea unui caracter la șir;
- Concatenarea a două șiruri;
- Găsirea unui subșir într-un șir dat.

Referințe utile: `strstr`.

Scrieți și codul aferent în `int main()` care să apeleze și să testeze funcționalitățile de mai sus.

18. Implementați o clasă `IntList`, care să rețină o listă simplu înlănțuită de numere întregi (de tip `int`), cu noduri alocate dinamic.

    Această clasă va avea definită în interiorul ei o clasă privată `Node`, care reprezintă un nod din listă. Fiecare nod va reține un număr întreg (valoarea acelui element din listă) și un pointer la următorul nod.

    Referințe utile: nested class;

    În clasa `IntList` este suficient să stocați un pointer la primul nod din listă pentru a o putea reține/accesa. Dacă doriți, puteți păstra și un pointer la ultimul nod din listă, pentru a adăuga mai ușor elemente la finalul ei.

    Implementați următoarele funcționalități:

    - Crearea unei liste vide (pointerii vor fi inițializați cu `NULL` / `nullptr`);
    - Adăugarea unui nou număr la începutul listei (metoda ar trebui să funcționeze atât cu o listă inițial vidă, cât și cu una care are deja elemente în ea);
    - Adăugarea unui nou număr la finalul listei (metoda ar trebui să funcționeze atât cu o listă inițial vidă, cât și cu una care are deja elemente în ea);
    - Eliberarea automată a memorie alocate;

      Indicație: implementați destructorul pentru clasa `Node`, astfel încât să dea `delete` la următorul nod din listă (dacă acesta nu este nul). În clasa `IntList`, va fi suficient să dați `delete` doar la primul nod, iar destructorii se vor apela recursiv, ștergând toată lista.
    - Crearea unei liste pornind de la un vector de numere întregi existent (i.e. să putem scrie `const int vector[] = { 1, 2, 3, 4 }; IntList lista(4, vector);`);
    - Copierea unei liste în altă listă (folosind constructorul de copiere, respectiv supraîncărcând operatorul `=`);

    (*) Mutarea unei liste în altă listă, lăsând vidă lista inițială;

    - Concatenarea a două liste (cele două liste primite ca parametru trebuie să rămână neschimbate);

- Accesarea elementului de pe poziția *i* (se va returna valoarea acestuia);
- Setarea valorii elementului de pe poziția *i*;
- Găsirea în listă a unui număr dat (se va returna dacă este sau nu prezent).